Algebra Relazionale

Data Definition Language (DDL)

Linguaggio per la definizione dello **schema** del database.

Permette operazioni come:

- Creazione, cancellazione e modifica di schemi di tabelle
- Creazione di viste e indici

Esempio:

```
create table Esami (
  Materia char(5),
  Candidato char(8),
  Voto int,
  Lode char(1),
  Data char(6)
);
```

Data Manipulation Language (DML)

Linguaggio degli **operatori del modello dei dati**, utilizzato per accedere e modificare i dati.

Consente operazioni di:

- Query (interrogazioni)
- Inserimento
- Cancellazione
- Aggiornamento (update)

Esempio:

```
select Candidato
from Esami
```

```
where Materia = "BDSI1" and Voto = 30;
```

Linguaggi Relazionali

- **Algebra relazionale**: insieme di operatori che agiscono su **relazioni** e producono come risultato **altre relazioni**.

  Non è utilizzata come linguaggio di interrogazione nei DBMS, ma come **rappresentazione interna** delle interrogazioni.
- **Calcolo relazionale**: linguaggio **dichiarativo** di tipo **logico**, dal quale è stato derivato l'**SQL (Structured Query Language)**.

Operatori Primitivi dell'Algebra Relazionale

- Ridenominazione
- Unione
- Differenza
- Proiezione
- Restrizione
- Prodotto

Notazione

- Nomi di relazioni: `R`, `S`, ...
- Nomi di attributi: `A`, `B`, `C`, `A1`, `A2`, ...
- Insiemi di attributi: `X`, `Y`, `X1`, ...
- `XY` è abbreviazione per `X ∪ Y`
- Relazione con ennuple `t1, t2, ..., tn` denotata con `{t1, t2, ..., tn}`
- Relazione vuota denotata con `{}`
- `tk.Ai` indica il valore di `Ai` nell'enunpla `tk`
- Se `X` è un sottoinsieme degli attributi di `t`, `t.X` o `t[X]` rappresenta l'enunpla ottenuta da `t` considerando solo gli attributi in `X`

- Se `R` e `S` hanno lo stesso attributo `Aj`, in caso di ambiguità:
  - `R.Aj` denota l'attributo `Aj` della relazione `R`
  - `S.Aj` denota l'attributo `Aj` della relazione `S`

Ridenominazione

- Notazione: `ρA ← B(R)`
- Si utilizza per **cambiare il tipo di una relazione**.
- **Attenzione:** modificare il nome di un attributo comporta una **modifica del tipo della relazione**.

Laureati

Id	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Laureati

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

`ρId ← Matricola(Laureati)`

Questa operazione rinomina l'attributo `Matricola` della relazione `Laureati` in `Id`.

Il risultato è una nuova relazione con la stessa struttura e dati di `Laureati`, ma con l'attributo rinominato.

Unione

`R ∪ S`

Restituisce la relazione ottenuta facendo l'unione delle ennuple di `R` con quelle di `S`.

Le due relazioni devono essere **dello stesso tipo** (stessi attributi e domini)

R

A	B	C
a1	b1	c1
a1	b1	c2
a2	b1	c1
a3	b1	c1

S

A	B	C
a1	b1	c1
a1	b2	c2

$R \cup S =$

A	B	C
a1	b1	c1
a1	b1	c2
a2	b2	c2
a2	b1	c1
a3	b1	c1

Differenza

`R - S`

Restituisce la relazione contenente le ennuple di `R` **non presenti** in `S`

R

A	B	C
a1	b1	c1
a1	b1	c2
a2	b1	c1
a3	b1	c1

S

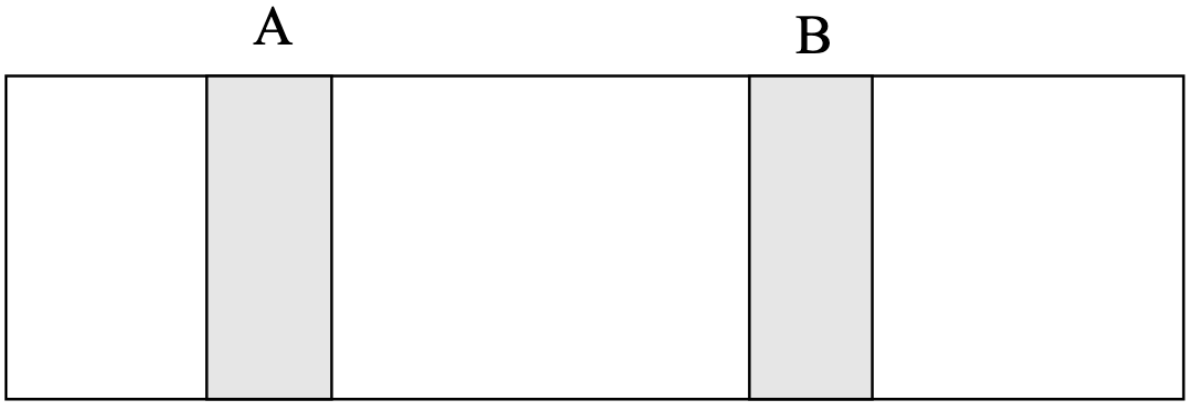
A	B	C
a1	b1	c1
a1	b2	c2

$R - S =$

A	B	C
a1	b1	c2
a2	b1	c1
a3	b1	c1

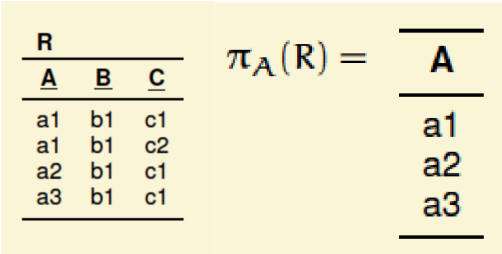
Proiezione

`πA1,...,An (R)`



Restituisce una relazione i cui elementi sono le ennuple di `R` **ristrette** agli attributi `A1,...,An`.

Poiché le relazioni sono **insiemi**, eventuali ennuple duplicate vengono eliminate automaticamente



Impiegati

Matricola	Cognome	Filiale	Stipendio
7309	Neri	Napoli	55
5998	Neri	Milano	64
9553	Rossi	Roma	44
5698	Rossi	Roma	64

$\pi_{\text{Cognome, Filiale}}$ (Impiegati)

Matricola	Cognome	Filiale	Stipendio
7309	Neri	Napoli	55
5998	Neri	Milano	64
9553	Rossi	Roma	44
5698	Rossi	Roma	64

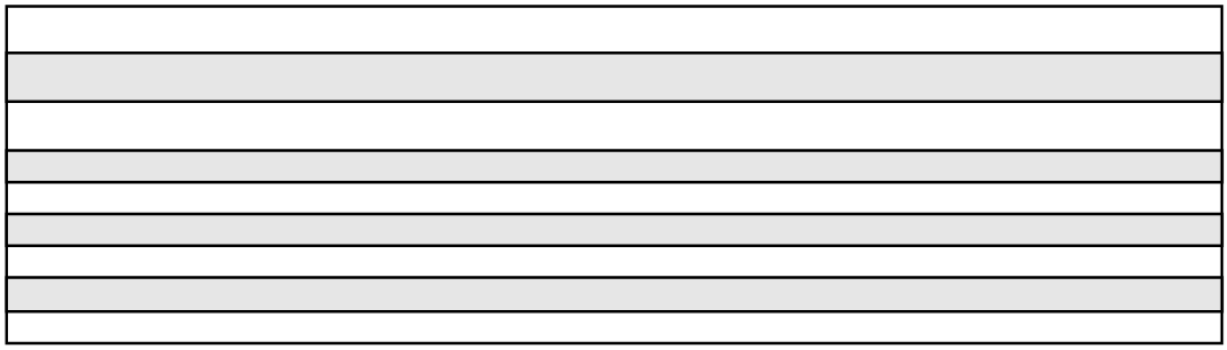
Cognome	Filiale
Neri	Napoli
Neri	Milano
Rossi	Roma

Cardinalità della Proiezione

- Una proiezione contiene **al più** tante ennuple quante ne ha l'operando.
- Se `X` è una **superchiave** di `R`, allora `πX(R)` contiene **lo stesso numero di ennuple** di `R`.
- Se `X` **non è una superchiave**, possono esistere valori ripetuti su quegli attributi, e quindi nel risultato appaiono **una sola volta**.

Restrizione (o Selezione)

`σφ(R)`



Restituisce una relazione dello stesso tipo di **R** i cui elementi sono la copia delle ennuple di **R** che soddisfano la condizione ϕ, definita come:

- `Ai θ Aj`, con **Ai** e **Aj** attributi di **R** e **θ** un operatore di confronto {<, >, =, ≠, ≤, ≥}
- `Ai θ c`, oppure `c θ Ai`, con **θ** operatore di confronto e **c** costante nel dominio di **Ai**
- Se ϕ e ψ sono formule, allora lo sono anche $\phi \wedge \psi$, $\phi \vee \psi$ e $\neg\psi$

R

A	B	C
a1	b1	c1
a1	b1	c2
a2	b1	c1
a3	b1	c1

$$\sigma_{A='a_1'}(R) =$$

A	B	C
a1	b1	c1
a1	b1	c2

esempio: *Impiegati che guadagnano più di 50 e lavorano a Milano*

Impiegati

Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
9553	Milano	Milano	44
5698	Neri	Napoli	64

$$\sigma_{\text{Stipendio} > 50 \text{ AND Filiale} = \text{'Milano'}}(\text{Impiegati})$$

Proprietà della Restrizione

- Produce un risultato con lo **stesso schema** dell'operando
- Contiene un **sottoinsieme** delle ennuple dell'operando
- La **condizione atomica** è vera solo per **valori non nulli**

Prodotto

$R \times S$

- Con **R** e **S** relazioni con attributi distinti, $R \times S$ è una relazione con elementi ottenuti **concatenando ogni ennupla di R con ogni ennupla di S**.
- La **concatenazione** di due ennuple è un'ennupla che contiene tutte le coppie delle ennuple che la compongono.
- La relazione risultante ha:
  - **Grado** = somma dei gradi degli operandi
  - **Cardinalità** = prodotto delle cardinalità degli operandi

esempio: *Trovare il nome e la matricola degli studenti della provincia di Pisa*

Studenti

Nome	Matricola	Provincia	Nascita
Isaia	71523	Pisa	1998
Rossi	67459	Lucca	1999
Bianchi	79856	Livorno	1998
Bonini	75649	Pisa	1999

Esami

Materia	Candidato*	Data	Voto
BD	71523	12/01/2018	28
BD	67459	15/09/2019	30
IA	79856	25/10/2020	30
BD	75649	27/06/2018	25
IS	71523	10/10/2019	18

$$\pi_{\text{Nome, Matricola}}(\sigma_{\text{Provincia = 'PI'}}(\text{Studenti}))$$

Nome	Matricola
Isaia	71523
Bonini	75649

Osservazione

- Il prodotto di due relazioni **non si usa da solo**, né per relazioni che non descrivono fatti in associazione tramite chiavi esterne.
- Le ennuple significative sono solo quelle con **uguali valori** di attributi come `Matricola` e `Candidato`.
  - Esempio: `R = σMatricola = Candidato(T)`
- Per concatenare ennuple in associazione, si **restringe sempre il prodotto** alle ennuple con valore uguale di chiave primaria e chiave esterna.
- Si introduce quindi un **operatore derivato**, detto **giunzione (join)**, che combina **prodotto** e **restrizione**.

Operatori Derivati

Operatori usati frequentemente come abbreviazioni di espressioni algebriche:

- **Intersezione**
- **Giunzione (Join)**
- **Giunzione Naturale (Natural Join)**
- **Semi-giunzione (Semijoin)**

Intersezione

$R \cap S$

- Restituisce la relazione ottenuta dall'intersezione delle ennuple di **R** con quelle di **S**, dove **R** e **S** sono dello stesso tipo.
- Equivalenza: $R \cap S = R - (R - S)$

R

A	B	C
a1	b1	c1
a1	b1	c2
a2	b1	c1
a3	b1	c1

S

A	B	C
a1	b1	c1
a1	b2	c2

$R \cap S =$

A	B	C
a1	b1	c1

Giunzione (Join / Equi-join / Inner-join)

$R \bowtie Ai = Aj\ S$

- Restituisce la relazione contenente le ennuple del **prodotto cartesiano R × S** in cui **Ai** (in R) e **Aj** (in S) hanno **valori uguali**.
- Vale l'equivalenza:

  $R \bowtie Ai = Aj\ S = \sigma Ai = Aj\ (R \times S)$

R

A	B	C
a1	b1	c1
a1	b1	c2
a2	b1	c1
a3	b1	c1

$T' = \rho_{A \leftarrow A'}(T) =$

A'	D
a1	d1
a2	d2

$R \underset{A=A'}{\bowtie} T' =$

A	B	C	A'	D
a1	b1	c1	a1	d1
a1	b1	c2	a1	d1
a2	b1	c1	a2	d2

Theta-Join

$R \bowtie_{\phi} S$

- L'**equi-join** è un caso particolare della **theta-join**, dove:

1. C'è un solo termine logico nella condizione ϕ
2. L'operatore è **l'uguaglianza (=)** tra due attributi di R e S

- In generale, ϕ è una **congiunzione ($\wedge$)** di termini logici di confronto `A1 θ A2` con **θ ∈ {=, >, <, ...}**

Giunzione Naturale (Natural Join)

`R ⋈ S = R ⋈A = A S = σA = A (R × S)`

- Restituisce la **relazione contenente le ennuple di R × S** con **attributi di uguali nomi** in **R** ed **S**, definiti sugli **stessi domini**.
- È una **abbreviazione dell'equi-join**, applicata a relazioni in cui l'associazione tra le ennuple è descritta da:
  - **Chiave esterna**
  - **Chiave primaria**

    costituite da **attributi con lo stesso nome**.

Osservazioni

- Se **R** e **S non hanno attributi comuni** → `R ⋈ S = R × S`
- Se **R** e **S hanno lo stesso schema** → `R ⋈ S = R ∩ S`
- Gli **operatori di giunzione** permettono la ricerca di ennuple **in corrispondenza con il meccanismo delle chiavi esterne**, e sono **gli unici** che offrono questa possibilità.

R

A	B	C
a1	b1	c1
a1	b1	c2
a2	b1	c1
a3	b1	c1

T

A	D
a1	d1
a2	d2

$R \bowtie T =$

A	B	C	D
a1	b1	c1	d1
a1	b1	c2	d1
a2	b1	c1	d2

Giunzione Naturale (Natural Join)

$R_1(A, B), R_2(B, C)$

Proprietà sulla Cardinalità

- Il join di R_1 e R_2 contiene un numero di ennuple **compreso fra zero e il prodotto** di $|R_1|$ e $|R_2|$

  ```
  D
  ```

- Se il join fra R_1 ed R_2 è **completo**, allora contiene **almeno** tante ennuple quante la relazione più grande:

  $$|R_1 \bowtie R_2| \geq \max(|R_1|, |R_2|)$$

- Se il join coinvolge una **chiave B di R_2**, allora:

  $$0 \leq |R_1 \bowtie R_2| \leq |R_1|$$

- Se il join coinvolge una **chiave B di R_2** e un **vincolo di integrità referenziale** tra l'attributo B di R_1 e la chiave di R_2, allora:

  $$|R_1 \bowtie R_2| = |R_1|$$

Semi-Giunzione (Semijoin)

$$R \ltimes S = \pi A_1, A_2, \ldots, A_m \, (R \bowtie S)$$

Descrizione

- La **semi-giunzione** restituisce **solo le ennuple di R** che partecipano alla **giunzione naturale** di R ed S.
- Gli attributi proiettati (**$A_1, A_2, \ldots, A_m$**) appartengono a **R**.

R

A	B	C
a1	b1	c1
a1	b1	c2
a2	b1	c1
a3	b1	c1

T

A	D
a1	d1
a2	d2

$R \rtimes T =$

A	B	C
a1	b1	c1
a1	b1	c2
a2	b1	c1

Osservazioni

- Non include gli attributi di **S** nel risultato.
- È utile per verificare **l'esistenza di corrispondenze** tra due relazioni senza duplicare dati.

Esempio concettuale:

> "Tutti gli studenti che hanno sostenuto almeno un esame."

(Si estraggono solo gli studenti presenti nella tabella degli esami, senza includere i dettagli degli esami stessi.)

Giunzione Esterna (External Join)

La **giunzione esterna** è una variante della giunzione che **mantiene anche le ennuple** che non trovano corrispondenza, **estendendole con valori nulli**.

Serve per **non perdere informazioni** quando alcune tuple non partecipano al join.

Tipologie di Giunzione Esterna

1. Giunzione Esterna Sinistra (Left Outer Join)

$$R \ ⟕ \ S$$

- Mantiene **tutte le ennuple di R** (il primo operando).

- Le ennuple di R che non trovano corrispondenze in S vengono **completate con valori nulli** per gli attributi di S.

---

2. Giunzione Esterna Destra (Right Outer Join)

$$R \rtimes S$$

- Mantiene **tutte le ennuple di S** (il secondo operando).
- Le ennuple di S che non trovano corrispondenze in R vengono **completate con valori nulli** per gli attributi di R.

---

3. Giunzione Esterna Completa (Full Outer Join)

$$R \ltimes\!\rtimes S$$

- Mantiene **tutte le ennuple di entrambi gli operandi**.
- Le ennuple non abbinate vengono **completate con valori nulli** negli attributi mancanti.

R

A	B	C
a1	b1	c1
a1	b1	c2
a2	b1	c1
a3	b1	c1

S

A	D
a1	d1
a2	d2
a4	d4

$R \bowtie S =$

A	B	C	D
a1	b1	c1	d1
a1	b1	c2	d1
a2	b1	c1	d2

$R \overleftrightarrow{\bowtie} S =$

A	B	C	D
a1	b1	c1	d1
a1	b1	c2	d1
a2	b1	c1	d2
a3	b1	c1	null
a4	null	null	d4

$R \overleftarrow{\bowtie} S =$

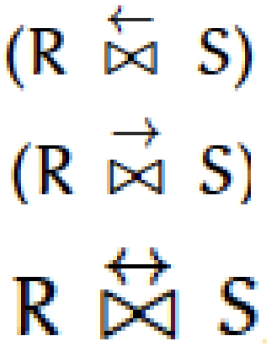
A	B	C	D
a1	b1	c1	d1
a1	b1	c2	d1
a2	b1	c1	d2
a3	b1	c1	null

$R \overrightarrow{\bowtie} S =$

A	B	C	D
a1	b1	c1	d1
a1	b1	c2	d1
a2	b1	c1	d2
a4	null	null	d4

Osservazione

- A differenza della **giunzione interna**, le giunzioni esterne **non eliminano** le tuple senza corrispondenze.
- Sono utili per **preservare l'informazione parziale** durante interrogazioni su dati incompleti.

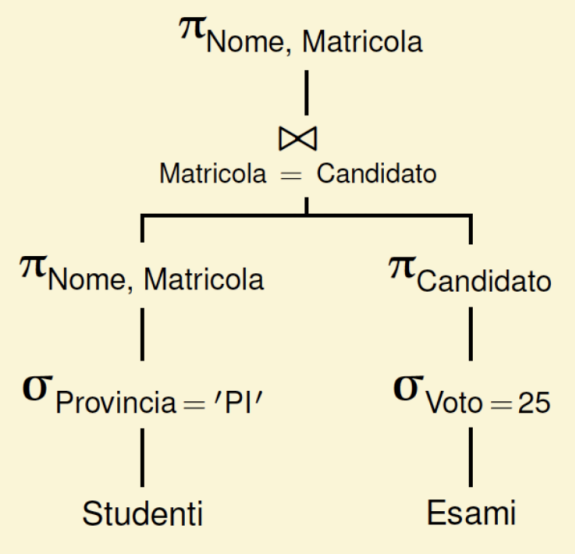


Self Join

- Supponiamo di considerare la seguente relazione **Genitori**
- **Obiettivo**: ottenere una relazione **Nonno–Nipote**
- In questo caso abbiamo bisogno di **utilizzare due volte la stessa tabella**

Rappresentazione ad Albero di Espressione Algebrica

$$
\begin{aligned}
&\pi_{\text{Nome, Matricola}} \, (\\
&\qquad \pi_{\text{Nome, Matricola}} \, (\sigma_{\text{Provincia} = \text{'PI'}} \, (\text{Studenti})) \\
&\qquad \bowtie_{\text{Matricola} = \text{Candidato}} \\
&\qquad \pi_{\text{Candidato}} \, (\sigma_{\text{Voto} = 25} \, (\text{Esami})) \\
&)
\end{aligned}
$$



Proprietà Algebriche degli Operatori Relazionali

Le **trasformazioni algebriche** sono utili per **ottimizzare le query**, riducendo anche di **ordini di grandezza** il costo di esecuzione.

L'idea principale è **riscrivere** le espressioni algebriche spostando **verso le foglie dell'albero di esecuzione** gli operatori di:

- **Proiezione (π)**
- **Restrizione (σ)**

In questo modo si **riduce la dimensione** dei risultati intermedi, migliorando l'efficienza complessiva della query.

Principi chiave:

- **Anticipare la restrizione (σ)**:
  - È l'operatore che, in generale, **riduce il numero di ennuple (righe)**.
  - Applicarla il prima possibile diminuisce la quantità di dati elaborati dai successivi operatori (come join o prodotto cartesiano).
- **Anticipare la proiezione (π)**:

- Riduce la **dimensione delle ennuple (colonne)**.
- Elimina eventuali **duplicati**.
- Applicarla precocemente evita di propagare colonne non necessarie nelle operazioni successive.

Proprietà Algebriche degli Operatori Relazionali

1. **Raggruppamento di restrizioni**

   Le restrizioni multiple possono essere combinate in un'unica operazione di selezione.

2. **Commutatività della restrizione e della proiezione**

   In certi casi, l'ordine tra selezione e proiezione può essere invertito senza alterare il risultato (se gli attributi coinvolti coincidono).

3. **Anticipazione della restrizione rispetto al prodotto e alla giunzione**

   Applicare la selezione prima del join o del prodotto riduce il numero di ennuple da elaborare.

4. **Raggruppamento di proiezioni**

   Più proiezioni consecutive possono essere sostituite da una sola proiezione con l'insieme minimo di attributi necessari.

5. **Eliminazione di proiezioni superflue**

   Una proiezione che non riduce gli attributi può essere eliminata.

6. **Anticipazione della proiezione rispetto al prodotto e alla giunzione**

   Applicare la proiezione prima di join o prodotto riduce la quantità di colonne propagate.

7. **Commutatività del prodotto e della giunzione**

   L'ordine degli operandi nel prodotto cartesiano o nella giunzione non altera il risultato.

8. **Associatività del prodotto e della giunzione**

La sequenza con cui vengono eseguiti più join o prodotti non influisce sul risultato finale.

9. **Commutatività degli operatori insiemistici**

   Per unione, intersezione e differenza simmetrica, l'ordine degli operandi non modifica il risultato.

10. **Associatività degli operatori insiemistici**

    L'ordine di esecuzione di più operazioni insiemistiche non altera il risultato finale.

11. **Anticipazione della restrizione rispetto agli operatori insiemistici**

    Applicare la selezione prima di un'operazione insiemistica riduce il numero di ennuple trattate.

12. **Anticipazione della proiezione rispetto agli operatori insiemistici**

    Applicare la proiezione prima di un'operazione insiemistica riduce il numero di attributi coinvolti.

Quantificatori Universali e Esistenziali

Ottenere **Matricola, Nome, Cognome, Voto** degli studenti:

- con (almeno un) voto maggiore di 28 (**quantificatore esistenziale**)
- che non hanno mai ottenuto un voto maggiore di 28 (**differenza**)
- che hanno ottenuto solo voti maggiori di 28 (**quantificatore universale**)

Quantificatore Esistenziale (∃)

Obiettivo: ottenere *Matricola, Nome, Cognome, Voto* degli studenti con **almeno un voto maggiore di 28**.

Descrizione:

Il quantificatore esistenziale indica che **esiste almeno un'istanza** per cui la condizione specificata è vera.

Nel contesto dei database relazionali, significa che esiste almeno una tupla (record) nella relazione che soddisfa la condizione indicata.

Esempio in algebra relazionale

$$\pi \text{ Matricola, Nome, Cognome, Voto } (\sigma \text{Voto}>28 \text{ (Studenti} \bowtie \text{Esami)})$$

Esempio in SQL

```
SELECT DISTINCT S.Matricola, S.Nome, S.Cognome, E.Voto
FROM Studenti S JOIN Esami E ON S.Matricola = E.Matricola
WHERE E.Voto > 28;
```

Risultato:

La query restituisce gli studenti che hanno sostenuto almeno un esame con voto superiore a 28

Matricola	Nome	Cognome
1	Luca	Rossi
2	Maria	Bianchi
3	Giorgio	Viola
4	Silvia	Neri
5	Enzo	Verdi

$\bowtie$

Matricola	Voto
1	25
3	30
2	23
1	29
4	29
1	26
5	30
4	30

$=$

Nome	Cognome	Matricola	Voto
Luca	Rossi	1	25
Giorgio	Viola	3	**30**
~~Maria~~	~~Bianchi~~	~~2~~	~~23~~
Luca	Rossi	1	**29**
Silvia	Neri	4	**29**
Luca	Rossi	1	26
Enzo	Verdi	5	**30**
Silvia	Neri	4	**30**

Trasformazione su Quantificatore Esistenziale

Il **quantificatore esistenziale (∃)** può essere espresso in algebra relazionale tramite operazioni di **giunzione** e **proiezione**, per identificare le entità che **soddisfano almeno una volta** una determinata condizione.

Esempio

Trovare la **Matricola**, **Nome**, **Cognome** degli studenti che hanno **almeno un voto maggiore di 28**.

Passaggi

1. **Selezione (Restrizione)**

   Si estraggono solo le ennuple della relazione *Esami* con voto > 28.

   ```
   σ Voto>28(Esami)
   ```

2. **Giunzione con Studenti**

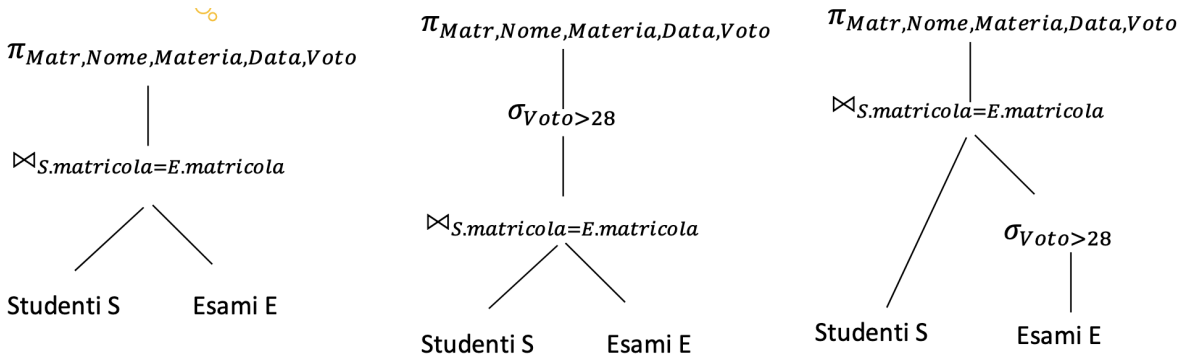
   Si collegano gli esami agli studenti corrispondenti tramite la chiave primaria/esterna *Matricola*

   ```
   Studenti ⋈Matricola σVoto>28(Esami)
   ```

3. **Proiezione**

   Si proiettano solo gli attributi desiderati (Matricola, Nome, Cognome).

   ```
   π Matricola, Nome, Cognome(Studenti ⋈Matricola σVoto>28(Esami))
   ```



Interpretazione

- La **restrizione** filtra i record di *Esami* che soddisfano la condizione (voto > 28).

- La **giunzione** associa tali record agli studenti che li hanno sostenuti.
- La **proiezione** restituisce solo le informazioni rilevanti degli studenti.

Equivalente in SQL

```
SELECT DISTINCT S.Matricola, S.Nome, S.Cognome
FROM Studenti S
JOIN Esami E ON S.Matricola = E.Matricola
WHERE E.Voto > 28;
```

Differenza

Il costrutto della **differenza** si utilizza per individuare le entità che **non soddisfano mai** una certa condizione, ovvero per applicare una forma **negata** del quantificatore esistenziale.

Esempio

Trovare la **Matricola**, **Nome**, **Cognome** degli studenti che **non hanno mai ottenuto un voto maggiore di 28**.

Matricola	Nome	Cognome
1	Luca	Rossi
2	Maria	Bianchi
3	Giorgio	Viola
4	Silvia	Neri
5	Enzo	Verdi

⋈

Matricola	Voto
1	25
3	30
2	23
1	29
4	29
1	26
5	30
4	30

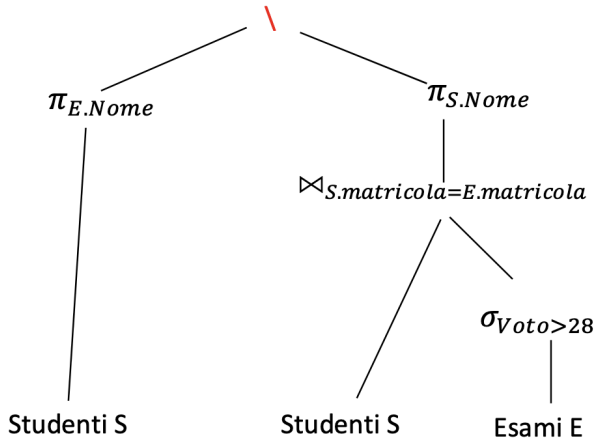
=

Nome	Cognome	Matricola	Voto
Luca	Rossi	1	25
Giorgio	Viola	3	**30**
Maria	Bianchi	2	23
Luca	Rossi	1	**29**
Silvia	Neri	4	**29**
Luca	Rossi	1	26
Enzo	Verdi	5	**30**
Silvia	Neri	4	**30**

Selezioniamo gli studenti che hanno preso voti >28

Faccio la differenza fra gli studenti e questo insieme

Albero Relazionale Differenze



Quantificatore Universale

Ottenere Matricola, Nome, Cognome, Voto degli studenti che hanno ottenuto solo voti maggiore di 28

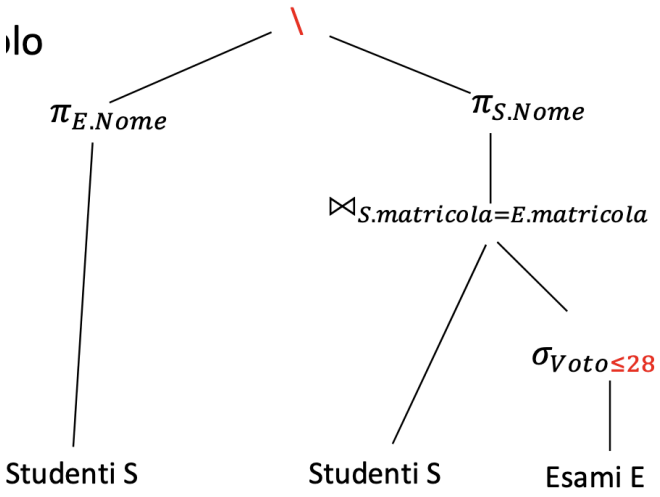
Matricola	Nome	Cognome
1	Luca	Rossi
2	Maria	Bianchi
3	Giorgio	Viola
4	Silvia	Neri
5	Enzo	Verdi

⋈

Matricola	Voto
1	25
3	30
2	23
1	29
4	29
1	26
5	30
4	30

=

Nome	Cognome	Matricola	Voto
Luca	Rossi	1	25
Giorgio	Viola	3	**30**
Maria	Bianchi	2	23
Luca	Rossi	1	**29**
Silvia	Neri	4	**29**
Luca	Rossi	1	26
Enzo	Verdi	5	**30**
Silvia	Neri	4	**30**



Raggruppamento

Definizione:

Il **raggruppamento** (grouping) è un'operazione dell'algebra relazionale che consente di applicare **funzioni di aggregazione** a insiemi di ennuple che condividono determinati valori di attributi.

Sintassi

```
{Ai} γ {fi}(R)
```

- **Ai** → attributi di raggruppamento (usati per formare i gruppi)

- **fi** → espressioni che utilizzano funzioni di aggregazione (`min`, `max`, `count`, `sum`, `avg`, ...)
- **R** → relazione di partenza

Procedura di Calcolo

1. **Partizionamento**:

   Le ennuple della relazione `R` vengono suddivise in **gruppi** in base ai valori uguali degli attributi `{Ai}`.

2. **Calcolo delle funzioni di aggregazione**:

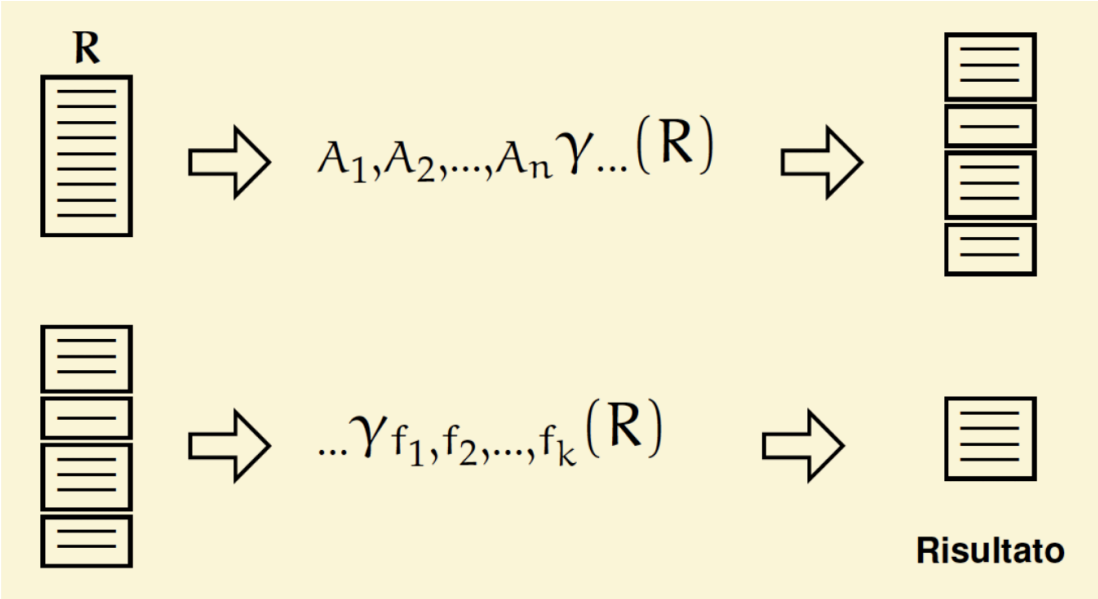
   Per ogni gruppo, vengono calcolate le espressioni `{fi}` (una per ciascuna funzione di aggregazione).

3. **Restituzione del risultato**:

   Per ogni gruppo si ottiene **una sola ennupla**, contenente:

   - i valori degli attributi di raggruppamento `{Ai}`
   - i risultati delle funzioni di aggregazione `{fi}`

$$\{A_i\}\gamma_{\{f_i\}}(R)$$



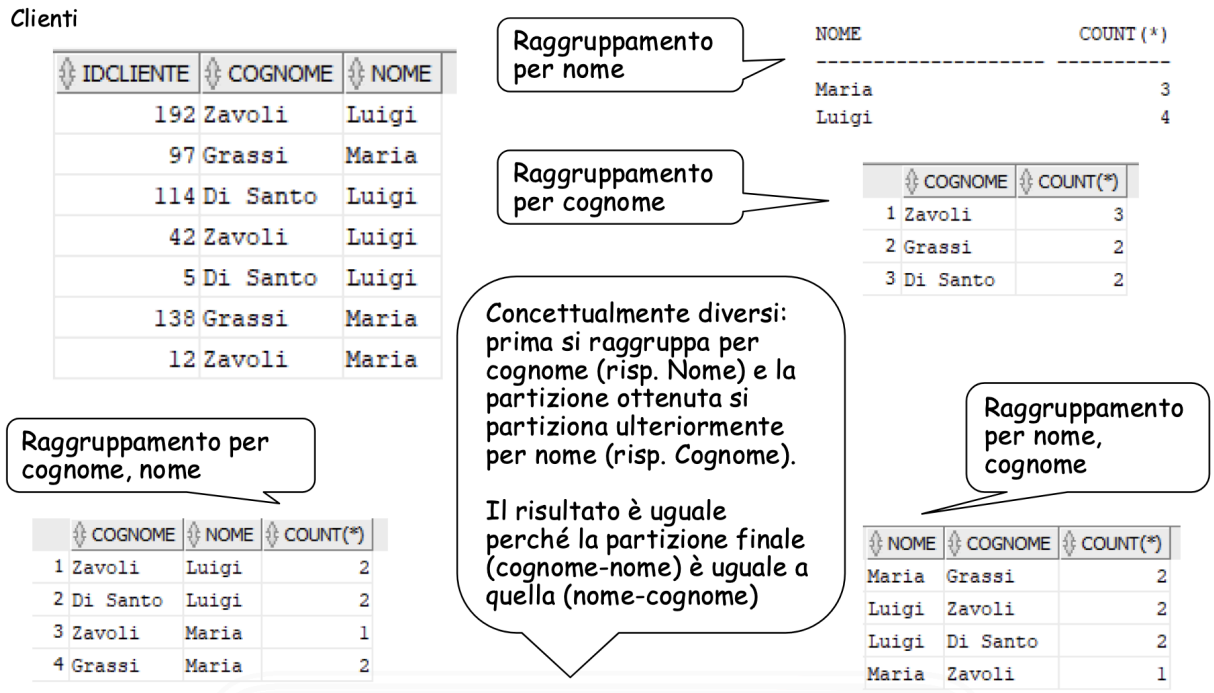
Per ogni candidato: numero degli esami, voto minimo, massimo e medio: ${\{\text{Candidato}\}}\gamma_{\{\text{count(*)},\text{min(Voto)},\text{max(Voto)},\text{avg(Voto)}\}}(Esami)$

Materia	Candidato	Voto	Docente
DA	1	20	10
LFC	2	30	20
MTI	1	30	30
LP	2	20	40

→

Materia	Candidato	Voto	Docente
DA	1	20	10
MTI	1	24	30
LFC	2	30	20
LP	2	20	40

Candidato	Count(*)	min(Voto)	max(Voto)	avg(Voto)
1	2	20	24	22
2	2	20	30	25



Raggruppamento: Misura e Dimensione

- **Posso raggruppare per voto?**

  Sì

- **Cosa succede se calcolo `MIN(voto)`?**

  Ottengo lo stesso valore del voto, perché tutti i record nel gruppo hanno lo stesso voto.

Esempio:

Materia	Candidato	Voto	Doc
Matematica	Rossi	28	Bianchi

Se raggruppo per *voto*, ogni gruppo conterrà tutti gli studenti che hanno preso quel voto.

Raggruppamento su Chiave Primaria

- **Posso raggruppare per Matricola?**

Sì, ma non ha molto senso, perché la chiave primaria identifica univocamente ogni record.

- **Cosa succede se calcolo `COUNT(*)`?**

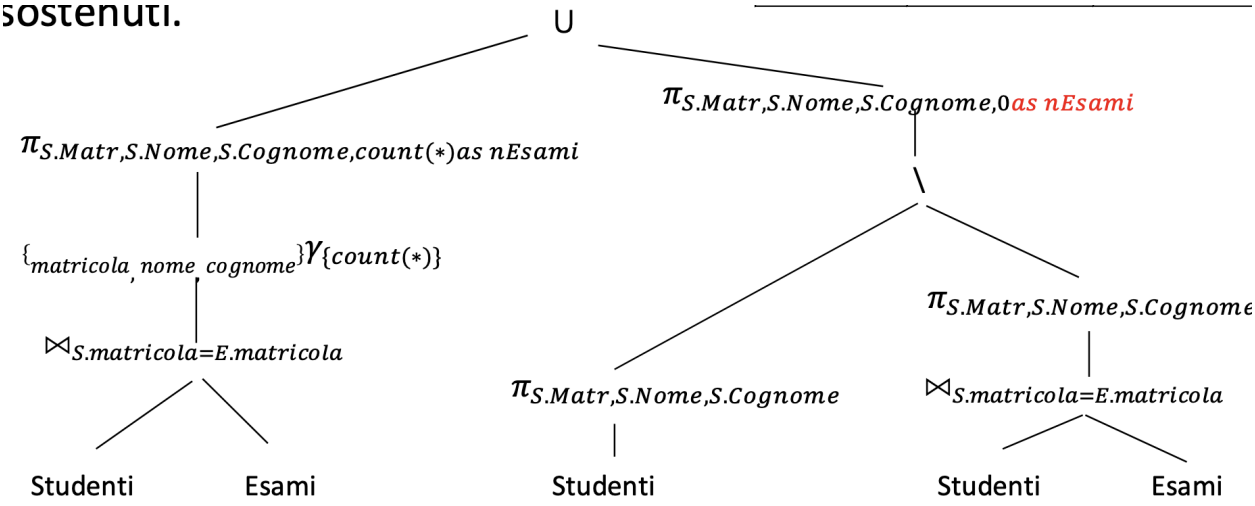
  Ottengo sempre 1, poiché ogni matricola compare una sola volta.

Esempio:

Nome	Matricola	Provincia	Anno
Rossi	123	Pisa	2023

Raggruppando per *Matricola*, ogni gruppo contiene un solo studente, quindi le funzioni di aggregazione restituiscono valori individuali.

sostenuti.



Proiezione Generalizzata

- Estende la proiezione con la possibilità di usare **costanti** o **espressioni aritmetiche** nella lista degli attributi.
- È possibile assegnare un'etichetta a un'espressione tramite l'operatore **AS**:

  $\pi e1\ AS\ ide1, e2\ AS\ ide2, \ldots (R)$
- Dove `e1`, `e2`, ... sono **espressioni aritmetiche** ottenute da costanti e attributi di `R`, e `ide1`, `ide2`, ... sono **etichette distinte**.

Proiezione Multi-Insiemistica (non insiemistica)

$\pi_b\{A_i\}(R)$

- Funziona come la proiezione, **ma senza eliminare i duplicati**.
- Il risultato è un **multi-insieme** di ennuple.

Ordinamento (non insiemistico)

$\tau Ai(R)$

- Ordina le ennuple di `R` in **ordine crescente o decrescente** rispetto agli attributi `A_i`.
- Sia la **proiezione multi-insiemistica** che l'**ordinamento** producono **multi-insiemi**, quindi possono essere usati **solo come radice di un albero logico**.

Calcolo Relazionale su Ennuple

- È un linguaggio che consente di definire il risultato di una **query** come **l'insieme delle ennuple** che soddisfano una certa **condizione** ϕ.

Confronto tra Algebra e Calcolo Relazionale

- L'**algebra relazionale** consente di costruire **espressioni annidate**, in cui gli operatori si applicano al risultato di altri operatori.
- Il **calcolo relazionale** ha una **struttura piatta**, ma permette **condizioni più complesse**.
- Entrambi hanno **lo stesso potere espressivo**: permettono di esprimere le stesse interrogazioni.

Verso un linguaggio unificato

Un linguaggio intermedio tra algebra e calcolo può essere ottenuto:

1. **Aggiungendo al calcolo** la possibilità di **annidare il costruttore di insiemi**.
2. **Aggiungendo all'algebra** la possibilità di includere **quantificatori** e **predicati di appartenenza** nelle condizioni di restrizione.

Risultato:

Un linguaggio in grado di combinare **interrogazioni annidate** e **condizioni logiche complesse**, come avviene in **SQL (***Structured Query Language***)**.